

IDENTIFYING KEY ENTITIES IN RECIPE DATA REPORT

BUSINESS OBJECTIVE

The goal of this assignment is to train a Named Entity Recognition (NER) model using Conditional Random Fields (CRF) to extract key entities from recipe data. The model will classify words into predefined categories such as ingredients, quantities and units, enabling the creation of a structured database of recipes and ingredients that can be used to power advanced features in recipe management systems, dietary tracking apps, or e-commerce platforms.

DATA DESCRIPTION

The given data is in JSON format, representing a structured recipe ingredient list with Named Entity Recognition (NER) labels. Below is a breakdown of the data fields:

[illegible]

Key	Description
input	Contains a raw ingredient list from a recipe.
pos	Represents the corresponding part-of-speech (POS) tags or NER labels, identifying quantities, ingredients, and units.

METHODOLOGY AND TECHNIQUES USED

This project follows a structured pipeline to perform Named Entity Recognition (NER) on recipe ingredient data. The methodology involves multiple stages from data preprocessing to model evaluation.

TECHNIQUES APPLIED

Natural Language Processing (NLP): We applied core NLP techniques such as:

- **Lexical Analysis:** Tokenizing the text into words.
- **Syntactic Analysis:** Using Part-of-Speech (POS) tagging and dependency parsing.
- **Named Entity Recognition (NER):** Identifying entities such as quantity, unit, and ingredient.

Machine Learning with CRF: A Conditional Random Field (CRF) model was used for sequence labeling, suitable for tasks like NER where context is important.

Exploratory Data Analysis (EDA): Basic data inspection and visualization helped us understand the dataset structure, token distribution, and label balance.

LIBRARY SETUP

The following Python libraries were used for various purposes:

- **Data Handling:** json, pandas, re, collections.Counter, fractions
- **NLP Processing:** spacy
- **Machine Learning & CRF:** sklearn_crfsuite, sklearn, joblib
- **Visualization:** matplotlib.pyplot, seaborn
- **Utility & Sampling:** random, numpy

The `sklearn-crfsuite` library plays a central role as it allows training CRF models using scikit-learn-like API, which is well-suited for structured output problems like NER.

DATA LOADING AND PREPROCESSING

- A custom function was written to load recipe data from a JSON file into a DataFrame.
- The input text and associated POS tags were split into lists of tokens (input_tokens and pos_tokens).
- We validated the integrity of the data by comparing the lengths of input and POS sequences using newly created columns (input_length and pos_length).

[illegible]

DATA CLEANING AND VALIDATION

To ensure clean and consistent data for training, the following steps were performed:

- A custom `unique_labels` function was created to extract all unique POS tags used in the dataset. The main labels identified were '**quantity**', '**unit**', and '**ingredient**'.
- We validated that each input sequence had a matching number of POS tags. Any rows where the lengths did not match (e.g., index 17, 27, 79, 164, 207) were considered invalid.
- These mismatched rows were removed to maintain alignment between the tokens and their labels.
- Finally, the `input_length` and `pos_length` columns were recalculated to reflect the cleaned dataset.

```
[14]: # Define a unique_labels function to checks for all the unique pos labels in the recipe & print it
def unique_labels(dataframe):
    unique_tags = set(label for sublist in dataframe['pos'] for label in sublist)
    return list(unique_tags)

unique_labels(ingredients_df)
```

```
[14]: ['unit', 'quantity', 'ingredient']
```

2.2.3 Provide the insights seen in the recipe data after validation [1 marks]

Provide the indexes that requires cleaning and formatting in the dataframe

[write your answer]

17,27,79,164,207

2.2.4 Drop the rows that have invalid data provided in previous cell [2 marks]

```
[15]: # drop the irrelevant recipe data
ingredients_df.drop(index=[17, 27, 79, 164, 207], inplace=True)
```

TRAIN-VALIDATION SPLIT

To evaluate the model effectively, the dataset was divided into **training (70%)** and **validation (30%)** sets using the `train_test_split` method with a fixed `random_state` of 42 for reproducibility.

- The `input_tokens` and `pos_tokens` were split into `X_train`, `X_val` and `y_train`, `y_val` respectively.
- All outputs were converted into NumPy arrays for faster processing and compatibility with the CRF model.

```
[18]: # split the dataset into training and validation sets

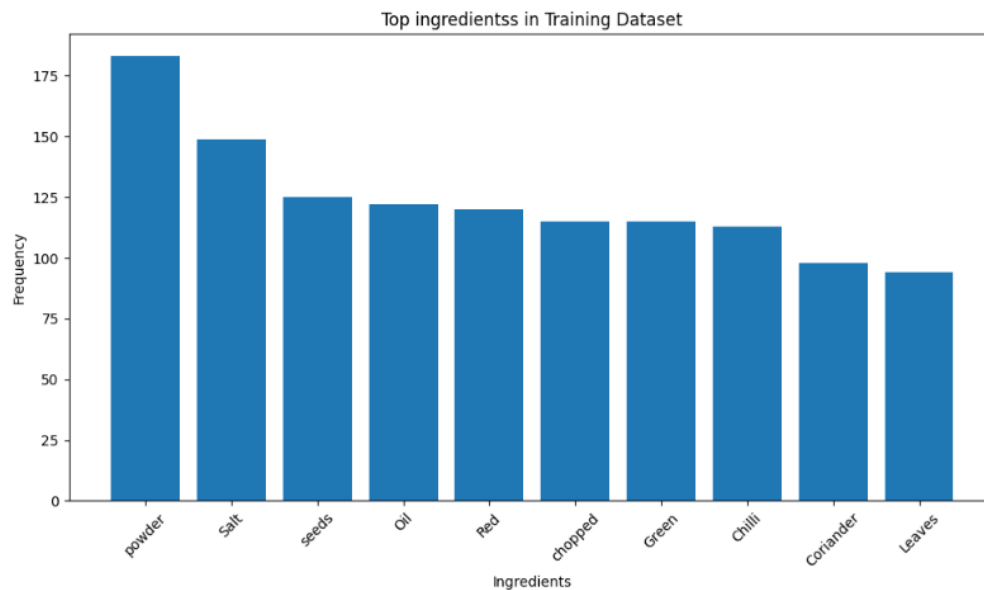
train_df, val_df = train_test_split(
    ingredients_df,
    test_size=0.3,
    shuffle=True,
    random_state=42
)
```

EXPLORATORY DATA ANALYSIS ON TRAINING DATA

Basic exploratory analysis was done on the training data to understand common ingredients and units used in the recipes:

Top Frequent Ingredients:

```
[34]: # plot the top frequent ingredients in training data
plot_top_items(top_items=top_items, label='ingredients', dataset_name=dataset_name)
```



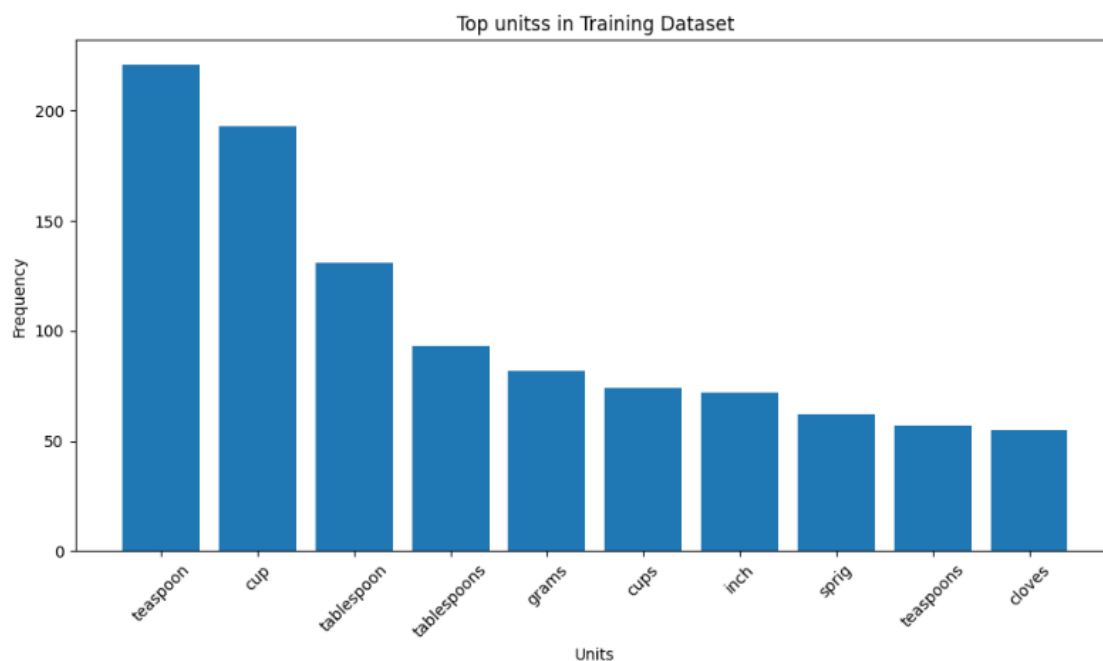
Top 5 Ingredients:

1. powder (183 occurrences)
2. Salt (149 occurrences)
3. seeds (125 occurrences)
4. Oil (122 occurrences)
5. Red (120 occurrences)

These results show that spices, seasonings, and common cooking ingredients dominate the dataset, reflecting typical Indian cuisine patterns.

Top Frequent Units:

```
[35]: # plot the top frequent units in training data
plot_top_items(top_items=top_units, label='units', dataset_name=dataset_name)
```



Top 5 Units:

1. teaspoon (221 occurrences)
2. cup (193 occurrences)
3. tablespoon (131 occurrences)
4. tablespoons (93 occurrences)
5. grams (82 occurrences)

The data shows that volume measurements (teaspoon, cup, tablespoon) are most common, followed by weight measurements (grams) and specialized units (inch, sprig, cloves).

FEATURE ENGINEERING AND WEIGHTING

To improve model performance and reduce class imbalance, the following steps were applied:

- The training label data (`y_train`) was flattened to create a single list of labels (`y_train_flat`). This allowed us to count the occurrence of each label and compute `label_counts`.
- Using these counts, **class weights** were calculated through the inverse frequency method. This gives more importance to less frequent labels like *quantity* and *unit*.
- To address the heavy dominance of the *ingredient* label, its weight was deliberately penalized to avoid model bias.
- A custom function `extract_features_with_class_weights` was defined to include these weights while generating features for both training and validation data.

```
[50]: # Compute class weights (inverse frequency method) by considering total_samples and label_counts
weight_dict = {
    label: total_samples / (3 * count)
    for label, count in label_counts.items()
}
```

```
[51]: weight_dict
```

```
[51]: {'quantity': 2.419727891156463,
      'unit': 2.923962186600904,
      'ingredient': 0.44548813325818776}
```

MODEL BUILDING AND TRAINING

A **Conditional Random Fields (CRF)** model was implemented using the `sklearn_crfsuite` library.

CRFs are particularly suitable for sequence labeling tasks like Named Entity Recognition (NER) because they take into account dependencies between neighboring labels, improving context-aware predictions.

The model was trained on the cleaned and feature-enhanced dataset using class weights to reduce the impact of label imbalance. The 'ingredient' label, being highly frequent, was penalized to ensure fair learning across all entity types.

```
[71]: # initialise CRF model with the specified hyperparameters and use weight_dict
from sklearn_crfsuite import CRF

# Create a CRF object with defined settings
crf_model = CRF(
    algorithm='lbfgs',          # Optimizer type
    c1=0.5,                    # Coefficient for L1 regularization
    c2=1.0,                    # Coefficient for L2 regularization
    max_iterations=100,        # Maximum number of optimization steps
    all_possible_transitions=True # Consider all tag transitions even if not seen during training
)

# train the CRF model with the weighted training data
crf_model.fit(X_train_weighted_features, y_train_labels)
```

```
[71]: CRF
CRF(algorithm='lbfgs', all_possible_transitions=True, c1=0.5, c2=1.0,
    max_iterations=100)
```

MODEL EVALUATION

The model's performance was assessed using:

- **Precision, Recall, and F1-Score** for each class
- **Confusion Matrix** for visualizing classification accuracy and errors
- **Overall Accuracy** on both training and validation datasets

This evaluation provided a comprehensive view of how well the model learned the data and generalized to unseen examples.

RESULTS AND PERFORMANCE

Training Set Results

- **Quantity:** Precision 99.59%, Recall 98.88%, F1-Score 99.23%
- **Unit:** Precision 97.90%, Recall 97.90%, F1-Score 97.90%
- **Ingredient:** Precision 99.47%, Recall 99.61%, F1-Score 99.54%
- **Overall Accuracy: 99.31%**
- **Total Tokens in Training Set: 7,114**

The training confusion matrix showed excellent classification, with very few misclassifications. Most confusion was between similar entity types like *unit* and *ingredient*, but errors were minimal.

```
[72]: # evaluate on the training dataset
from sklearn_crfsuite import metrics

# Predict Labels on the training data
y_train_pred = crf_model.predict(X_train_weighted_features)

# Evaluate using classification report
print("Classification Report on Training Set:")
print(metrics.flat_classification_report(y_true=y_train_labels, y_pred=y_train_pred))
```

```
Classification Report on Training Set:
              precision    recall  f1-score   support

ingredient      0.99        1.00        1.00        5323
quantity         1.00        0.99        0.99         980
unit             0.98        0.98        0.98         811

accuracy              0.99              0.99        7114
macro avg           0.99        0.99        0.99        7114
weighted avg        0.99        0.99        0.99        7114
```

```
[73]: # specify the flat classification report by using training data for evaluation
from sklearn_crfsuite import metrics

# Predict Labels on training data
y_train_pred = crf_model.predict(X_train_weighted_features)

# Generate classification report
train_report = metrics.flat_classification_report(
    y_true=y_train_labels,
    y_pred=y_train_pred,
    labels=["quantity", "unit", "ingredient"],
    digits=4
)

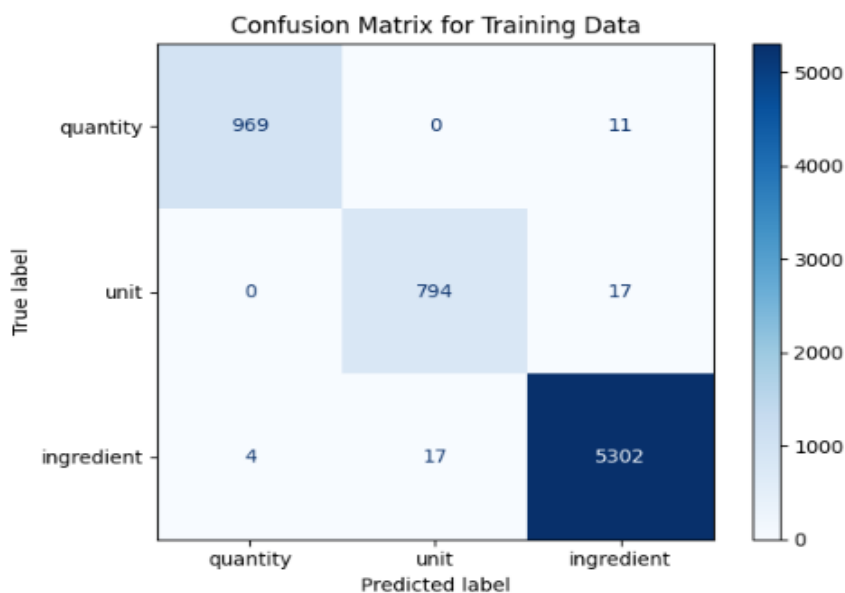
# Print the report
print("Flat Classification Report on Training Data:\n")
print(train_report)
```

```
Flat Classification Report on Training Data:

              precision    recall  f1-score   support

quantity      0.9959      0.9888      0.9923         980
unit           0.9790      0.9790      0.9790         811
ingredient     0.9947      0.9961      0.9954        5323

accuracy              0.9931              0.9931        7114
macro avg         0.9899      0.9880      0.9889        7114
weighted avg      0.9931      0.9931      0.9931        7114
```



Validation Set Results

- **Quantity:** Precision 99.02%, Recall 98.05%, F1-Score 98.53%
- **Unit:** Precision 97.37%, Recall 93.02%, F1-Score 95.14%
- **Ingredient:** Precision 98.64%, Recall 99.57%, F1-Score 99.10%
- **Overall Accuracy: 98.54%**
- **Total Tokens in Validation Set: 2,876**

The validation confusion matrix confirmed strong generalization. Most *ingredient* tokens were correctly predicted, while some confusion was observed in *unit* labels, especially in ambiguous measurement contexts.

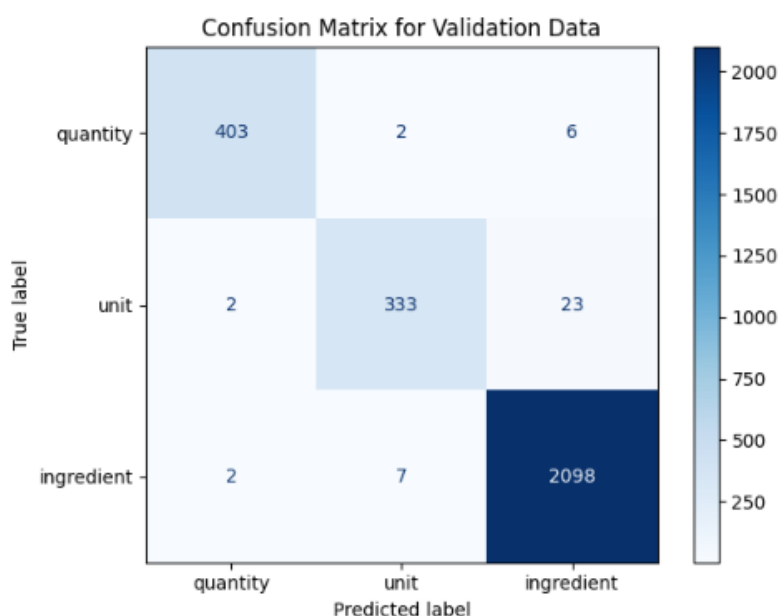
```
[78]: # specify flat classification report
from sklearn_crfsuite import metrics

# Generate classification report on validation data
val_report = metrics.flat_classification_report(
    y_true=y_val_labels,
    y_pred=y_val_pred,
    labels=["quantity", "unit", "ingredient"], # Specify consistent label order
    digits=4
)

# Print the report
print("Flat Classification Report on Validation Data:\n")
print(val_report)
```

Flat Classification Report on Validation Data:

	precision	recall	f1-score	support
quantity	0.9902	0.9805	0.9853	411
unit	0.9737	0.9302	0.9514	358
ingredient	0.9864	0.9957	0.9910	2107
accuracy			0.9854	2876
macro avg	0.9834	0.9688	0.9759	2876
weighted avg	0.9853	0.9854	0.9853	2876



ERROR ANALYSIS

A total of **42 errors** were found in the **2,876 validation tokens**, yielding an **error rate of 1.46%**.

Error Distribution:

- **Quantity:** Very few errors due to consistent patterns (403/411 correct)
- **Unit:** Most errors occurred here (26/358 incorrect), often due to ambiguous measurements
- **Ingredient:** Performed best overall, with only minor misclassifications (2098/2107 correct)

Key Error Patterns:

- Ambiguous tokens serving as both ingredient and unit
- Multi-word ingredient names being split or mislabeled
- Context-dependent measurement terms
- Unseen or novel ingredient names not present in training data.

```
[84]: # Analyse errors found in the validation data by each Label
      # and display their class weights along with accuracy
      # and display the error dataframe with token, previous token, next token, true label, predicted label and context

      # Count errors by true label
      error_by_label = error_df['true_label'].value_counts().to_dict()

      # Total tokens per label in validation set
      from collections import Counter

      # Flatten true labels from validation set
      y_val_flat = [label for seq in y_val_labels for label in seq]
      val_label_counts = Counter(y_val_flat)

      # Compute per-label accuracy and show class weights
      print("Error Analysis by Label:\n")
      print(f"{'Label':<12}{'Errors':<10}{'Total':<10}{'Accuracy':<12}{'Class Weight'}")
      print("-" * 55)

      for label in val_label_counts:
          total = val_label_counts[label]
          errors = error_by_label.get(label, 0)
          acc = (total - errors) / total
          weight = weight_dict.get(label, 1.0)
          print(f"{'label':<12}{'errors':<10}{'total':<10}{'acc:<12.4f}{'weight:<.4f}'")
```

Error Analysis by Label:

Label	Errors	Total	Accuracy	Class Weight
quantity	8	411	0.9805	2.4197
unit	25	358	0.9302	2.9240
ingredient	9	2107	0.9957	0.0445

KEY ACHIEVEMENTS

- Built a high-accuracy NER system for recipe-style text
- Reached **98.54% validation accuracy** through robust CRF modeling and balanced training
- Applied **effective class weighting** to reduce bias toward dominant labels
- Established a clean, reusable **pipeline for entity extraction** in recipes.

SUMMARY

The CRF model effectively extracted recipe entities with over 98% accuracy, demonstrating strong generalization and minimal error. Through strategic feature engineering and class balancing, the system proved robust for practical NER applications in culinary text.